22AIE205 – Introduction to Python

Python Multithreaded Programming

Running multiple tasks concurrently with Python's threading module

1. Process & Thread
2. The GIL
3. threading module
4. ThreadPoolExecutor

What is a Process?

An executing program, managed by the operating system

Program Code

The instructions that the CPU executes

Data

Variables, buffers, and its own workspace

Execution Context

The current state of the running process

A Process (PID: 1042)

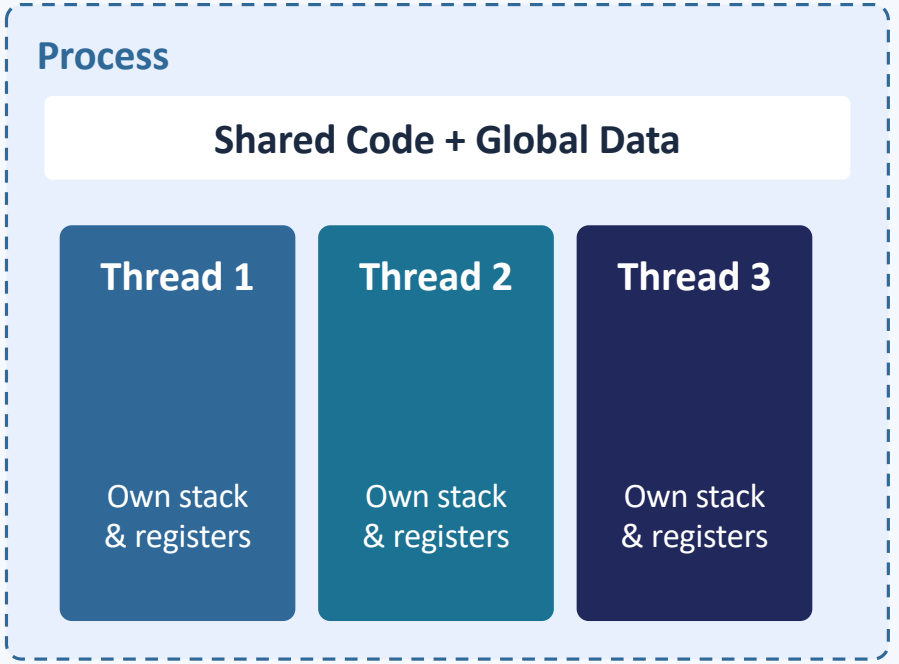
- Code
- Data (variables, buffers)
- Execution context (state)

Each process gets its own memory space — isolated from other processes.

What is a Thread?

The smallest unit of execution inside a process

- A process can contain multiple threads
- Threads share the same code and global data
- Each thread keeps its own registers and local variables (its own stack)
- Think of a thread as a lightweight sub-task inside a process



How Multithreading Works

On a single CPU core, Python switches rapidly between threads

Key Idea: Context Switching

- CPU rapidly switches between threads, giving each a small time slice.
- This makes threads appear to run at the same time (concurrency).
- Background tasks run without freezing the main program.
- True parallel CPU execution needs multiple cores — see GIL, next slide.

Single Core Timeline



CPU → time →

Thread 1 and Thread 2 take turns on the one core, switching so fast it looks simultaneous.

The GIL — Global Interpreter Lock

Why Python threads don't run Python code on multiple cores at once

What it means

- CPython allows only one thread to execute Python bytecode at any instant — protected by a single lock, the GIL.
- So threads never truly run Python code in parallel, even on a multi-core machine.
- Threads still help a lot for I/O-bound work: file reads, network calls, DB queries — the GIL is released while waiting.
- For CPU-bound work (heavy computation), use the multiprocessing module instead, which uses separate processes.

I/O-bound tasks

Waiting on network, disk, or user input

→ use threading

CPU-bound tasks

Heavy computation (math, image processing)

→ use multiprocessing

Single-threaded vs Multithreaded

Single-Threaded Process

Task A

Task B (waits)

Only one task runs at a time — the next task waits until the current one finishes completely.

Multithreaded Process

Task A
(own stack)

Task B
(own stack)

Both tasks run concurrently, sharing the same code and data but keeping separate stacks/registers.

Threading in Python — 4 Steps

Python's built-in threading module makes this simple

1

Import the module

```
import threading
```

2

Create threads

```
t1 =
threading.Thread(
target=func1)
t2 =
threading.Thread(
target=func2)
```

3

Start threads

```
t1.start()
t2.start()
```

4

Wait for completion

```
t1.join()
t2.join()
```

Example: Two Threads Running Together

One thread prints numbers, another prints letters — both run concurrently

```
import threading
import time

def print_numbers():
    for i in range(1, 4):
        print(f"Number: {i}")
        time.sleep(1)

def print_letters():
    for ch in ['A', 'B', 'C']:
        print(f"Letter: {ch}")
        time.sleep(1)

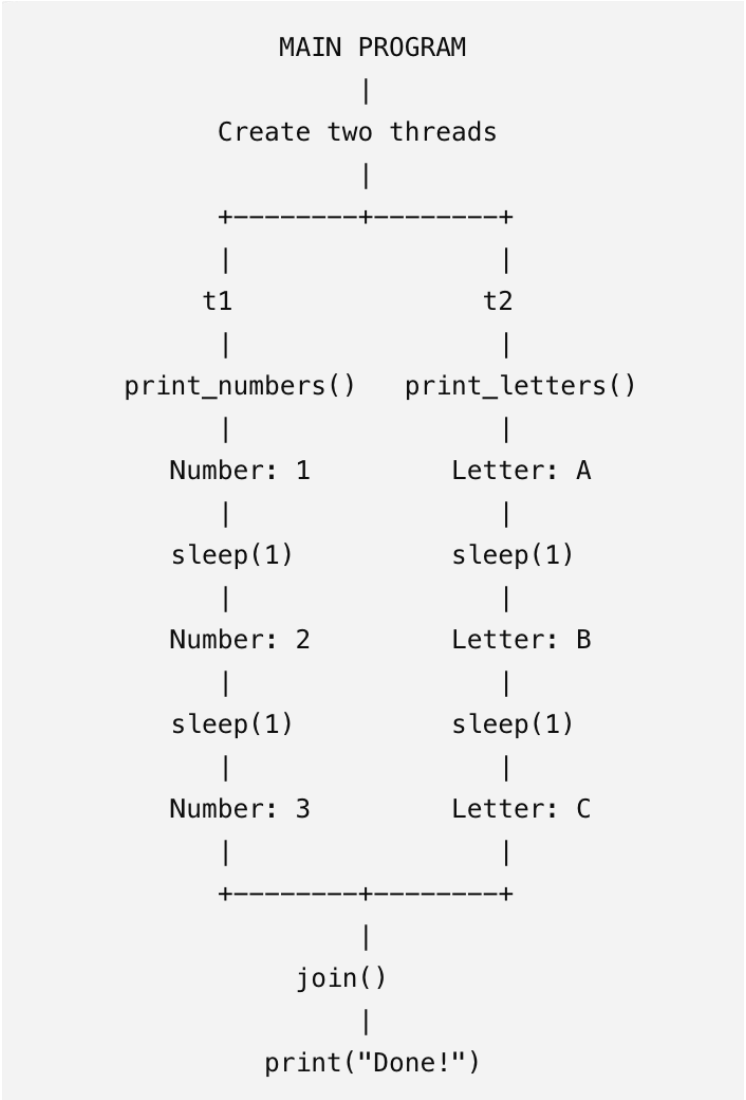
t1 = threading.Thread(target=print_numbers)
t2 = threading.Thread(target=print_letters)

t1.start()
t2.start()
t1.join()
t2.join()
print("Done!")
```

What happens

- Both threads start almost together and run in parallel time slices.
- Since execution order isn't fixed, output may interleave — e.g. Number: 1, Letter: A, Number: 2 ...
- join() makes the main program wait until both threads finish before printing "Done!".

```
Possible output:
Number: 1   Letter: A   Number: 2
...
```




```
Number: 1
Letter: A
Number: 2
Letter: B
Number: 3
Letter: C
Done!
```

```
Letter: A
Number: 1
Number: 2
Letter: B
Letter: C
Number: 3
Done!
```

ThreadPoolExecutor — Simpler Thread Management

concurrent.futures.ThreadPoolExecutor manages a pool of worker threads for you

```
from concurrent.futures import ThreadPoolExecutor

def process_order(order_id):
    print(f"Processing order {order_id}")

# Pool of 2 worker threads
with ThreadPoolExecutor(max_workers=2) as pool:
    # Submit orders to run concurrently
    pool.submit(process_order, 101)
    pool.submit(process_order, 102)
```

Explanation

- Creates a pool with 2 worker threads, ready to pick up tasks.
- Each submit() hands one order off to a free worker thread.
- The pool manages start/join for you and shuts down automatically at the end of the with block.

Use ThreadPoolExecutor when you have many similar short tasks (e.g. processing several orders, downloading several files) — it's simpler and safer than managing Thread objects by hand.

Process vs Thread

Process

- Independent program in execution, own memory space
- Identified by a unique Process ID (PID)
- States: new → ready → running → waiting → terminated
- Processes talk to each other via Inter-Process Communication (IPC)

Thread

- Smallest unit of execution, lives inside a process
- Identified by a Thread ID (TID); shares memory with siblings
- States: new → runnable → running → waiting → terminated
- Threads communicate faster since they share memory directly

Similarities Between Threads and Processes

1 Units of Execution

Both are execution units managed by the OS as part of process management.

2 OS Scheduling

Both are scheduled by the OS for fair CPU allocation and can be preempted.

3 Own Execution Context

Each has its own program counter, CPU registers, and stack space.

4 Creation & Communication

Both can spawn children at runtime, use IPC, and release resources when they end.

Key Takeaways

1. A process is an independent program; a thread is a lightweight execution unit inside it, sharing code and data.
2. Python's threading module: create with Thread(), start(), then join() to wait for completion.
3. The GIL lets only one thread run Python bytecode at a time — great for I/O-bound tasks, not for CPU-bound ones.
4. ThreadPoolExecutor gives you a managed pool of worker threads for many similar short tasks.
5. For CPU-heavy parallel work, reach for the multiprocessing module instead.